Программ хангамжийн инженерчлэлийн тусгай сэдвүүд (F.CSA303)

Д. Батмөнх

2026 оны 3-р сарын 19

Агуулга

1-р хичээл **2**

2-р хичээл **2**

3-р хичээл **2**

4-р хичээл **4**

5-р хичээл **5**

A Лабораторийн ажлын файлын зохион байгуулалт **8**

B Linux суулгах **9**

C IDE суулгах **10**

D LaTeX орчин суулгах **10**

E Монгол үгийн алдаа шалгагч ашиглах **11**

F Git ашиглах **11**

G Unit test хийх жишээ **15**

H Бичил шалгалтын бусад жишээ код **18**

I Лабораторийн хавсралт **21**

1-р хичээл

Хугацаа: 1/29–2/12.

1. Лабораторийн ажлын орчин бэлтгэх (хавсралтыг ашиглана):
   - Linux,
   - Git,
   - LibreOffice,
   - LaTeX
2. Бичил шалгалт хэрхэн хийхийг хавсралтаас ойлгох

2-р хичээл

Хугацаа: 2/12–2/26. Даалгаврын гүйцэтгэл кодоо код удирдах системд хадгална.

1. IntelliJ IDEA засварлагч ашиглахдаа Google Java Style ашиглах тохиргоо хийнэ.
2. Лабораторийн хавсралт №1 кодын хувьд DRY, KISS, SRP, OCP зарчмуудын аль нь хангагдаагүйг тодорхойлж (аль зарчим хэддүгээр мөрт), засварла.
3. Мөн хавсралт №2 кодын хувьд дээрх зарчмуудын аль нь хангагдаагүйг тодорхойлж, засварла.
4. Лабораторийн хавсралт №3 кодод байгаа асуудлуудыг тодорхойлон, засварлаж, бичил шалгалтыг бич.

3-р хичээл

Хугацаа: 2/26–3/12.

1. Энэхүү даалгаварт программд учирч болох алдааг боловсруулаагүйгээс үүдэлтэй бодит амьдралд тулгарч болох асуудлыг жишээгээр хийлгэж байгаа бөгөөд дараах `lab3.1.zip` файлыг зураг

дээр дарж татаж аваад, задална. Дараа нь terminal нээгээд дараах командыг бичиж, ажиллуулна[1].

```
chmod +x start_atm.sh
./start_atm.sh
```

Ингэмэгц дэлгэцэд дараах мэдээлэл харагдана:

```
====== ATM CONTROLLER ======

> [1] ATM-1 PID=13867 RUNNING
  [2] ATM-2 PID=13868 RUNNING
  [3] ATM-3 PID=13869 RUNNING

1/2/3 -> ATM сонгох | тоо (>=10) -> дүн | Ctrl+C -> гарах

Withdrawal amount (default ATM=1):
```

Withdrawal amount гэдэгт 10-аас 10,000 хүртэл утга оруулж болно. Санах ойд нэг кодыг 3 зэрэг ажиллуулж байгаа бөгөөд хэрэв тооноос бусад утга оруулбал, 1 worker нь алдаа өгч зогсоно. Санах ойд хэдэн worker ажиллаж байгааг шалгахдаа дараах командыг ашиглаж болно:

```
ps aux | grep ATMProcess
```

Энэхүү мэдээллийг ATM Controller хэсэгт мөн үзүүлэх болно. Тэгвэл өгөгдсөн `*.java` файлыг засаж, тооноос бусад утга оруулахад буруу утга оруулсныг сануулж, программын хэвийн ажиллагааг үргэлжлүүлэхээр кодыг зас.

2. Функцийн Switch Statement агуулсан тохиолдол нь clean code байж чадах уу? Ямар тохиолдолд чадах вэ? Чадахгүй бол ямар шалтгааны улмаас? Түүнийг хэрхэн сайжруулах хэрэгтэй вэ?

3. Дараах классын аль нь илүү ортогональ загвартай байна вэ? Яагаад?

[1] Жич, Java JDK (Java Development Kit) суусан байх ёстой. macOS Preview дээр хавсаргасан файл татагдахгүй бол `F.CSA303_Lab_Work.pdf` файлаа Firefox дээр нээгээд татаарай.

```
class Split1 {
    public Split1(InputStreamReader rdr) { ...
    public void readNextLine() throws IOException { ...
    public int numFields() { ...
    public String getField(int fieldNo) { ...
}

class Split2 {
    public Split2(String line) { ...
    public int numFields() { ...
    public String getField(int fieldNo) { ...
}
```

4. Modeless болон Modal харилцан цонхны аль нь ортогональ загвар байж чадах вэ?

5. Процедур хэл болон объект технологийн аль нь ортогональ загвар вэ?

4-р хичээл

Хугацаа: 3/12–3/26.

1. Дараах кодууд нь Деметрийн хуулийг зөрчиж байгаа эсэхийг тогтоо:

   Java хэл дээр:

   ```
   public void showBalance(BankAccount acct) {
       Money amt = acct.getBalance();
       printToScreen(amt.printFormat());
   }
   ```

   C++ хэл дээр:

   ```
     void processTransaction(BankAccount acct, int) {
       Person *who;
       Money amt;

       amt.setValue(123.45);
       acct.setBalance(amt);
   ```

```
    who = acct.getOwner();
    markWorkflow(who->name(), SET_BALANCE);
  }
```

Java хэл дээр:

```
 public class Colada {
    private Blender myBlender;
    private Vector myStuff;

    public Colada() {
      myBlender = new Blender();
      myStuff = new Vector();
    }

    private void doSomething() {
      myBlender.addIngredients(myStuff.elements());
    }
  }
```

2. Python хэлний Optional ямар үүрэгтэй вэ? Түүнийг ашиглахын ач холбогдол юу вэ? Жишээгээр тайлбарла.

3. None буюу null утга буцаахын сөрөг тал нь юу байж болохыг жишээгээр тайлбарла.

4. None буюу null утга буцаахаас хэрхэн зайлсхийхийг жишээгээр харуул.

5-р хичээл

Хугацаа: 3/26–4/09.

1. Abstract class болон Interface хоёрын ялгааг бич.

2. Test Double болон Mock Object гэж юу вэ? Жишээгээр тайлбарла.

3. Хяналтын урвуу тогтолцоо болон хамаатуулах нэвтрэлт хоёрын хамаарлыг тайлбарлаж, эдгээр нь нэг үүргийн зарчмыг (SRP) хэрхэн хангахыг тайлбарла.

4. Dependency Injection ашиглахын давуу тал юу вэ? Жишээгээр тайлбарла.

5. Command-Line Argument Parser жишээ бичнэ. Ингэхдээ OCP зарчим хангагдахаар дараах сонголтуудыг args ашиглан гүйцэтгэнэ:

   - Өгөгдсөн файлын мөр бүр зөвхөн нэг үг (латин ба кирилл байж болно) агуулах бөгөөд түүнийг цагаан толгойн дараалаар эрэмбэл
   - Цагаан толгойн урвуу дараалаар эрэмбэл
   - Давхардсан үгийг хасаж эрэмбэл (мөн урвуугаар)
   - Том үсгээр эхэлснийг эхэнд нь жижиг үсгээр эхэлснийг дараа нь байрлуул
   - Дээрх функц бүрд бичил шалгалт хий

A Лабораторийн ажлын файлын зохион байгуулалт

```
F.CSM301_lab/
|-- .git/
|-- .gitignore
|-- README.md
|
|-- lab01/
|   |-- README.md
|   |-- assignment.md
|   |
|   |-- java/
|   |   |-- src/
|   |   |   \-- main/java/com/lab01/
|   |   |       \-- Main.java
|   |   |-- src/
|   |   |   \-- test/java/com/lab01/
|   |   |       \-- MainTest.java
|   |   |-- pom.xml
|   |   \-- README_java.md
|   |
|   |-- python/
|   |   |-- src/
|   |   |   \-- main.py
|   |   |-- tests/
|   |   |   \-- test_main.py
|   |   |-- requirements.txt
|   |   \-- README_python.md
|   |
|   |-- cpp/
|   |   |-- src/
|   |   |   \-- main.cpp
|   |   |-- tests/
|   |   |   \-- test_main.cpp
|   |   |-- CMakeLists.txt
|   |   \-- README_cpp.md
|   |
|   |-- data/
```

```
|   |   |-- input/
|   |   |   |-- test1.txt
|   |   |   |-- test2.txt
|   |   |   \-- ...
|   |   \-- expected/
|   |       |-- test1.txt
|   |       |-- test2.txt
|   |       \-- ...
|   |
|   |-- docs/
|   |   |-- lab01_report.md
|   |   |-- setup.md
|   |   \-- F.CSM301_Lab_Work.pdf
|   |
|   \-- scripts/
|       |-- build_all.sh
|       |-- build_java.sh
|       |-- build_python.sh
|       |-- build_cpp.sh
|       |-- run_tests.sh
|       |-- test_java.sh
|       |-- test_python.sh
|       \-- test_cpp.sh
|
|-- ...
...
```

B Linux суулгах

1. GNOME хэрэглэгчийн орчин бүхий Debian системийг татаж авна: `debian-live-13.x.x-amd64-gnome.iso`

2. Bootable USB бэлтгэхийн тулд Etcher эсвэл Ventoy программыг татан суулгаад, ажиллуулна. Улмаар `iso` файлаа зааж, Bootable USB бэлтгэнэ. Ventoy ашиглаж байгаа бол bootable дискээ эхэлж бэлтгээд (`https://www.ventoy.net/en/doc_start.html` зааврын дагуу) дараа нь `*.iso` файлаа хуулаад, ашиглахад бэлэн болно.

3. Дараа нь компьютероо дахин эхлүүлж, `BIOS/UEFI` тохиргоо уруу ороод (ихэвчлэн `F2`, `F12`, `ESC`, `DEL`), `Boot order` хэсэгт `USB`-г хамгийн

түрүүнд ачаалахаар тохируулна. Хэрэв `Fast Boot` асаатай бол энэ тохиргоог унтраана. Мөн `SATA Operation` тохиргоог `AHCI` болгосон байна.

4. Ийнхүү бэлтгэсэн USB дискээ ачаалж, Debian систем суулгах үйлдлийг эхлүүлэх бөгөөд суулгах явцад `Partition disk` хэсэгт `Install alongside Windows` гэж сонгоод (Windows системийг устгалгүй Linux системтэй хамт буюу dual boot горимоор ажиллана), `GRUB Bootloader` суулгана.

C IDE суулгах

```
sudo apt install wget default-jdk -y
cd /tmp

wget -O jetbrains-toolbox.tar.gz \
"https://data.services.jetbrains.com/products/download?code=TBA&platform=linux"

tar -xvzf jetbrains-toolbox.tar.gz
sudo mv jetbrains-toolbox-*/ /opt/jetbrains-toolbox

/opt/jetbrains-toolbox/bin/jetbrains-toolbox
```

Анх ажиллуулахад `Toolbox update` хийх бөгөөд дараа нь `Application` цэс дотор `shortcut` үүснэ. Түүнийг ашиглаад IntelliJ IDEA суулгана.

Суулгасны дараа оюутны имэйлээ JetBrains Products for Learning бүртгүүлээд үнэгүй ашиглах эрхээ авна.

D LaTeX орчин суулгах

Дараах заавар нь LaTeX системийг бүрэн бус хувилбараар (хэмжээ бага) суулгах бөгөөд код засварлагчаар TeXstudio программыг сонгон суулгах болно:

```
sudo apt install texlive-latex-base \
                 texlive-latex-recommended \
                 texlive-xetex \
                 texlive-lang-cyrillic \
                 fonts-noto \
                 texstudio
```

E Монгол үгийн алдаа шалгагч ашиглах

Линукс системд LibreOffice, TeXstudio программууд дээр монгол үгийн алдаа шалгагчийг хэрхэн ашиглах зааврыг `https://zuv.bichig.dev/` сайтаас харна уу.

F Git ашиглах

SSH key үүсгэхийн тулд `terminal` программаа нээгээд дараах командыг биелүүлнэ:

```
sudo apt update
sudo apt install git -y

git config --global user.name "Нэрээ оруулна"
git config --global user.email "Имэйлээ оруулна"

ssh-keygen -t ed25519 -C "your_email@example.com"
ssh-add ~/.ssh/id_ed25519
```

Улмаар SSH түлхүүрийг GitLab-д нэмнэ. Дараа нь, GitLab-д шинээр project[2] үүсгээд түүнийгээ компьютертоо дараах байдлаар татаж авна:

```
cd ~/Documents
mkdir Projects
cd Projects
git clone https://github.com/username/my-project.git
```

Git ашиглах заавар

.gitignore

```
# ==============================================================================
# OS / Editor junk
# ==============================================================================
.DS_Store
Thumbs.db
ehthumbs.db
Icon?
```

[2]Үнэгүй ашиглах нөхцөлд 1 repository буюу project-н дээд хязгаар нь 10ГБ бөгөөд хичээл бүрийн тоогоор repository үүсгэх боломжтой

```
*.swp
*.swo
*~

# VSCode / IntelliJ / Eclipse
.vscode/
.idea/
*.iml
*.ipr
*.iws
.classpath
.project
.settings/
.metadata

# ==============================================================================
# Python
# ==============================================================================
__pycache__/
*.py[cod]
*$py.class

# Virtual environments
.venv/
venv/
ENV/
env/
env.bak/
venv.bak/

# Packaging / build
build/
dist/
.eggs/
*.egg-info/
*.egg

# Test / coverage
.pytest_cache/
.tox/
```

```
.nox/
.coverage
.coverage.*
htmlcov/
nosetests.xml
coverage.xml
*.cover
.hypothesis/

# ==============================================================================
# Java (Maven / Gradle)
# ==============================================================================
# Build output
target/
**/build/
!src/**/build/

# Maven
pom.xml.tag
pom.xml.releaseBackup
pom.xml.versionsBackup
pom.xml.next
release.properties
dependency-reduced-pom.xml

# Gradle
.gradle/
!gradle-wrapper.jar

# Compiled classes / packages
*.class
*.jar
*.war
*.ear
*.nar
*.zip
*.tar.gz

# JVM crash logs
hs_err_pid*
```

```
replay_pid*

# ==============================================================================
# C++ (CMake / build)
# ==============================================================================
# Object / binary files
*.o
*.obj
*.so
*.dylib
*.dll
*.a
*.lib
*.exe
*.out
*.app

# CMake build dirs
CMakeFiles/
CMakeCache.txt
cmake_install.cmake
Makefile
CTestTestfile.cmake
_deps/
cmake-build-*/

# ==============================================================================
# Project-specific
# ==============================================================================
# Test results / reports
test_results/
test_reports/
coverage_reports/
*_report.xml
*_results.json

# Generated docs (keep /docs in repo)
docs/generated/
*_generated.pdf
```

```
# Ignore local scripts output if any
scripts/*.log
scripts/*.tmp
```

G Unit test хийх жишээ

Шалгалтад суурилсан хөгжүүлэлт буюу Test Driven Development (TDD) зарчмыг баримтлан хэрхэн код бичихийг алхам алхмаар үзүүлье.

Дараах unit testing framework ашиглана:

- C++ бол Catch2
- Java бол JUnit
- Python бол unittest

Бичил шалгалтын код бичнэ, жишээлбэл, Java хэл дээр:

```
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

public class CalculatorTest {

    @Test
    public void testAddTwoPositiveNumbers() {

        Calculator calculator = new Calculator();

        int result = calculator.add(3, 4);
        assertEquals(7, result, "3 + 4 нь 7-тэй тэнцүү байх ёстой");
    }
}
```

Бичил шалгалтын кодыг ажиллуулмагц алдаа заана, учир нь Calculator класс байхгүй. Бүтээгдэхүүний кодод энэ классыг бичнэ:

```
public class Calculator {

    public int add(int a, int b) {
        return 7;
    }
  }
```

Бичил шалгалтыг дахин ажиллуулмагц амжилттай давах болно. Шинэ бичил шалгалт нэмнэ:

```
public class CalculatorTest {

    @Test
    public void testAddTwoPositiveNumbers() {

        Calculator calculator = new Calculator();

        int result = calculator.add(3, 4);
        assertEquals(7, result, "3 + 4 нь 7-тэй тэнцүү байх ёстой");
    }

    @Test
    public void testAddPositiveAndNegativeNumbers() {

      Calculator calculator = new Calculator();
      int result = calculator.add(3, -4);
      assertEquals(-1, result, "3 + (-4) нь -1-тэй тэнцүү байх ёстой");
    }
}
```

Бичил шалгалтыг дахин ажиллуулмагц унах болно, учир нь бүтээгдэхүүний кодод тогтмол тоо 7 буцаасан байгаа. Бичил шалгалтын кодоо сайжруулна (refactoring):

```
public class Calculator {

    public int add(int a, int b) {
        return a + b;
    }
}
```

Бичил шалгалтыг дахин ажиллуулмагц давах болно. Шинэ бичил шалгалтын код нэмье:

```
public class CalculatorTest {

    @Test
    public void testAddTwoPositiveNumbers() {
```

```
        Calculator calculator = new Calculator();

        int result = calculator.add(3, 4);
        assertEquals(7, result, "3 + 4 нь 7-тэй тэнцүү байх ёстой");
    }

    @Test
    public void testAddPositiveAndNegativeNumbers() {

      Calculator calculator = new Calculator();
      int result = calculator.add(3, -4);
      assertEquals(-1, result, "3 + (-4) нь -1-тэй тэнцүү байх ёстой");
    }

    @Test
    public void testMultiplyTwoPositiveNumbers() {
      Calculator calculator = new Calculator();

      int result = calculator.multiply(3, 4);
      assertEquals(12, result, "3 * 4 нь 12-той тэнцүү байх ёстой");
    }
}
```

Бичил шалгалтыг дахин ажиллуулмагц унах болно. Шинэ бичил шалгалтын код нэмье:

```
public class Calculator {

    public int add(int a, int b) {
        return a + b;
    }

    public int multiply(int a, int b) {
        return a * b;
    }
}
```

Бичил шалгалтыг дахин ажиллуулмагц амжилттай давах болно.

H Бичил шалгалтын бусад жишээ код

Python хэл дээр:

```
import unittest


def insertion_sort(A, n):
    for i in range(1, n):
        key = A[i]
        j = i - 1
        while j >= 0 and A[j] > key:
            A[j + 1] = A[j]
            j = j - 1
        A[j + 1] = key


class TestInsertionSort(unittest.TestCase):
    def test_insertion_sort(self):
        lst = [12, 3, 7, 9, 14, 6, 11, 2]
        insertion_sort(lst, len(l))
        self.assertEqual([2, 3, 6, 7, 9, 11, 12, 14], lst)


if __name__ == "__main__":
    unittest.main()
```

Java хэл дээр:

```
// InsertionSort.java
public class InsertionSort {
    public void insertionSort(int[] A, int n) {
        for (int i = 1; i < n; ++i) {
            int key = A[i];
            int j = i - 1;
            while (j >= 0 && A[j] > key) {
                A[j + 1] = A[j];
                j--;
            }
            A[j + 1] = key;
        }
    }
}

// InsertionSortTest.java
import org.junit.jupiter.api.Assertions;
import org.junit.jupiter.api.Test;

public class InsertionSortTest {
    @Test
    public void sortArray() {
        int[] array = {12, 3, 7, 9, 14, 6, 11, 2};
        InsertionSort insertionSortTests = new InsertionSort();
        insertionSortTests.insertionSort(array, array.length);
        Assertions.assertArrayEquals(new int[]{2, 3, 6, 7, 9, 11, 12, 14}, array);
    }
}
```

C++ хэл дээр шалгахдаа

```
// CMakeLists.txt
cmake_minimum_required(VERSION 3.25)
project(InsertionSort)

Include(FetchContent)

FetchContent_Declare(
        Catch2
        GIT_REPOSITORY https://github.com/catchorg/Catch2.git
        GIT_TAG        v3.4.0 # or a later release
)

FetchContent_MakeAvailable(Catch2)

add_executable(InsertionSort main.cpp)
target_link_libraries(InsertionSort PRIVATE Catch2::Catch2WithMain)

set(CMAKE_CXX_STANDARD 20)
```

бэлтгээд

```
cmake .
make
```

командаар төсөлд шаардлагатай файлуудаа үүсгэнэ.

```
// main.cpp
#include <catch2/catch_test_macros.hpp>
#include <catch2/matchers/catch_matchers_vector.hpp>
#include <vector>

void insertionSort(std::vector<int> A, int n) {
    for (int i = 1; i < n; ++i) {
        int key = A[i];
        int j = i - 1;
        while (j >= 0 && A[j] > key) {
            A[j + 1] = A[j];
            j = j - 1;
        }
        A[j + 1] = key;
    }
}


TEST_CASE (
"Array sort"
)
 {
    std::vector<int> array = {12, 3, 7};
    insertionSort(array, sizeof(array));
    REQUIRE_THAT(array, Catch::Matchers::UnorderedEquals(std::vector<int>{3, 7, 12}));
}
```

I Лабораторийн хавсралт

№1

```
public class PostService {
    public void deletePost(Long postId, Long userId) {
        Post post = postRepository.findById(postId);

        User user = userRepository.findById(userId);
        if (user.getRole().equals("ADMIN")) {
            postRepository.delete(post);
            return;
        }
```

```
        if (post.getAuthorId().equals(userId)) {
            postRepository.delete(post);
            return;
        }

        throw new UnauthorizedException("Cannot delete this post");
    }
}

public class CommentService {
    public void editComment(Long commentId, Long userId, String newText) {
        Comment comment = commentRepository.findById(commentId);

        User currentUser = userRepository.findById(userId);
        boolean isAdmin = currentUser.getRole().equals("ADMIN");
        boolean isOwner = comment.getUserId().equals(userId);

        if (!isAdmin && !isOwner) {
            throw new UnauthorizedException("Cannot edit this comment");
        }

        comment.setText(newText);
        commentRepository.save(comment);
    }
}
```

№2

```
public class UserRegistrationService {
    public void registerUser(String email, String password) {
        if (!email.contains("@")) {
            throw new InvalidEmailException("Email must contain @");
        }
        if (email.length() < 5) {
            throw new InvalidEmailException("Email too short");
        }
        if (email.length() > 50) {
            throw new InvalidEmailException("Email too long");
        }
```

```
        User user = new User(email, password);
        userRepository.save(user);
    }
}

public class PartnerService {
    public void addPartner(String companyEmail, String companyName) {
        if (!companyEmail.contains("@")) {
            throw new InvalidEmailException("Email must contain @");
        }
        if (companyEmail.length() < 5) {
            throw new InvalidEmailException("Email too short");
        }
        if (companyEmail.length() > 50) {
            throw new InvalidEmailException("Email too long");
        }

        Partner partner = new Partner(companyEmail, companyName);
        partnerRepository.save(partner);
    }
}
```

№3

```
public class ProductService {

    // "Энгийн" бүтээгдэхүүний жагсаалт авах функц
    public List<Product> getProductsByCategory(String category) {
        // 1. Database-аас татах
        List<Product> products = productRepository.findByCategory(category);

        // 2. "Үзсэн тоог нэмэх"
        for (Product p : products) {
            p.setViewCount(p.getViewCount() + 1);
        }
        productRepository.saveAll(products);

        // 3. "Мөн популяр бүтээгдэхүүн шинэчлэх"
        updatePopularProducts(category, products);
```

```
        // 4. "Cache-д хадгалах"
        cacheService.put("products_" + category, products);

        // 5. "Хайлтын түүхэнд нэмэх"
        searchHistoryService.recordSearch(category, products.size());

        return products;
    }

    private void updatePopularProducts(String category, List<Product> products) {
        PopularityData data = new PopularityData();
        data.setCategory(category);
        data.setProductCount(products.size());
        data.setLastAccessed(new Date());
        popularityRepository.save(data);
    }
}
```